

AD9082/AD9081/AD9988/AD9986 Python Example Application User Guide

MPG SW System Apps

Exported on 06/20/2023

Table of Contents

1	Hardware Requirements.....	4
2	Run Environment Tools Requirements.....	5
3	Release Package Components	6
4	Setup Python Environment	7
5	Configure Example Application For Target Hardware & Support Files	10
6	Run Example Application	12
7	Python Example Application- Query APIs.....	15
7.1	Overview of sample Debug Scripts	15
7.2	Creating your own sample Debug Scripts.....	15
7.3	Note on Reading Back data from API function in Python Environment via eRPC .NET DLL	17
8	Customizing or Creating a Use-case	19
9	AD9081 eRPC Python Client - Server Demo	21
9.1	Python with eRPC Client - Server Demo	21
9.2	Python with eRPC Client - Server Demo with Eclipse and Pydev	23

This is the user guide for getting started with the Python Demo Application.

This tool is a Python based Example Application targeting the AD9081/AD9082/AD9988/AD9986 Evaluation Platform.

In addition to the Python example application requires some Windows Runtime support files and libraries for the AD9081/AD9082 Evaluation Board.

This document illustrates the steps to get started using the Python Example Application using a BASH Environment on Windows.

The source code and libraries are provided in the release package the end user may port to Windows Python environment they might require and use the source code as a reference to develop their own tests etc.

The AD9081/AD9082/AD9988/AD9986 Python Example Application uses a number of python packages and so we recommend created a

python virtual environment with a tool such as PIPENV. The steps below will guide you through this setup.

The AD9081/AD9082 Python Demo Application show examples of how to perform the following operation

1. Detect the MicroZed
2. Configure the ADS9-V2 FPGA image
3. Copy the erpc server to the MicroZed and start it
4. Configure the MxFE device
5. Display JESD link status
6. Download a vector to the ADS9-V2
7. Capture data from the the ADS9-V2
8. Plot time domain data.
9. Option to repeat capture

1 Hardware Requirements

- **AD9081/A9082 Evaluation Daughter Board**
- **ADS9 Evaluation Platform**

2 Run Environment Tools Requirements

- **Window Environment**
- **Python Version 3.7.5**
- **Python Packet Manager such as PIP**
- **Virtual Environment Tools such as pipenv**

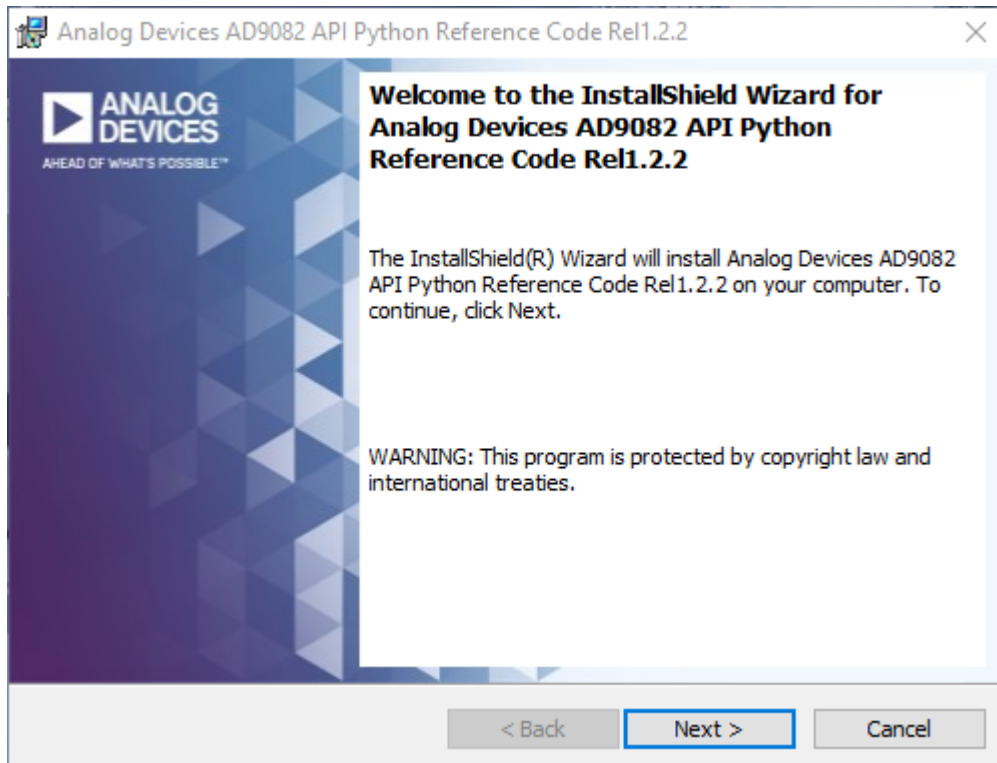
3 Release Package Components

ADI Component Folder	Description	Notes
doc	License Information and User Guide Documentation	
adxxx_python_eval_app	Python Source Code Example App	Main Python Application source code folder
erpc_pkgs	PC-Evaluation Board Client -Server	Includes Latest erpc client server files • .Net DLL AD9082/AD9081/AD9988/AD9986 client for Python Application • AD9082/AD9081/AD9988/AD9986 server for Microzed.
fpga images	ADS9 Board Image	Includes Latest ADS9 FPGA Binary
data_control_libs	Data Capture and Data Playback Libraries	.Net DLLs for Data Capture and Playback Libraries for AD9082/AD9081/AD9988/AD9986 Evaluation Platform

4 Setup Python Environment

Ref : [Python Basics \(pythontutorial.net\)](https://www.pythontutorial.net/python-basics/)¹ for help with Python.

- Extract Python Source Code Package by running the installer.

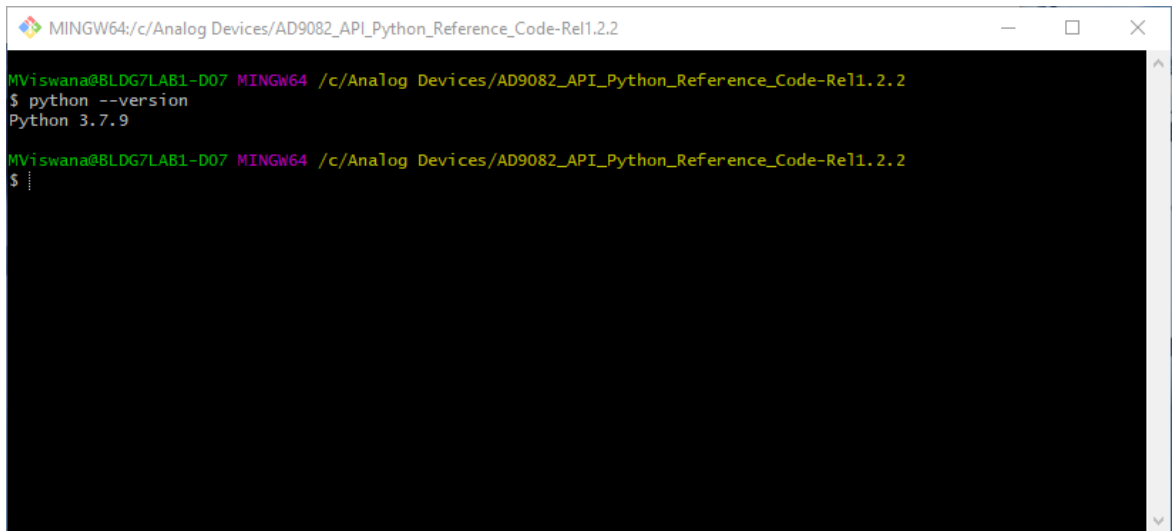


- Navigate to the C:\Analog Devices\AD9082_API_Python_Reference_Code-Rel1.2.2 Directory and review the example python source code package.

```
MINGW64:/c/Analog Devices/AD9082_API_Python_Reference_Code-Rel1.2.2
MViswana@BLDG7LAB1-D07 MINGW64 /c/Analog Devices
$ cd AD9082_API_Python_Reference_Code-Rel1.2.2/
MViswana@BLDG7LAB1-D07 MINGW64 /c/Analog Devices/AD9082_API_Python_Reference_Code-Rel1.2.2
$ ls
ad9082_python_eval_app/  data_control_libs/  doc/  erpc_pkgs/  fpga_images/
MViswana@BLDG7LAB1-D07 MINGW64 /c/Analog Devices/AD9082_API_Python_Reference_Code-Rel1.2.2
$ |
```

¹ <https://www.pythontutorial.net/python-basics/>

- Ensure you have Python 3.7.5. If you have a version higher than 3.7.5, install 3.7.5 for compatibility with Python Virtual Environment.



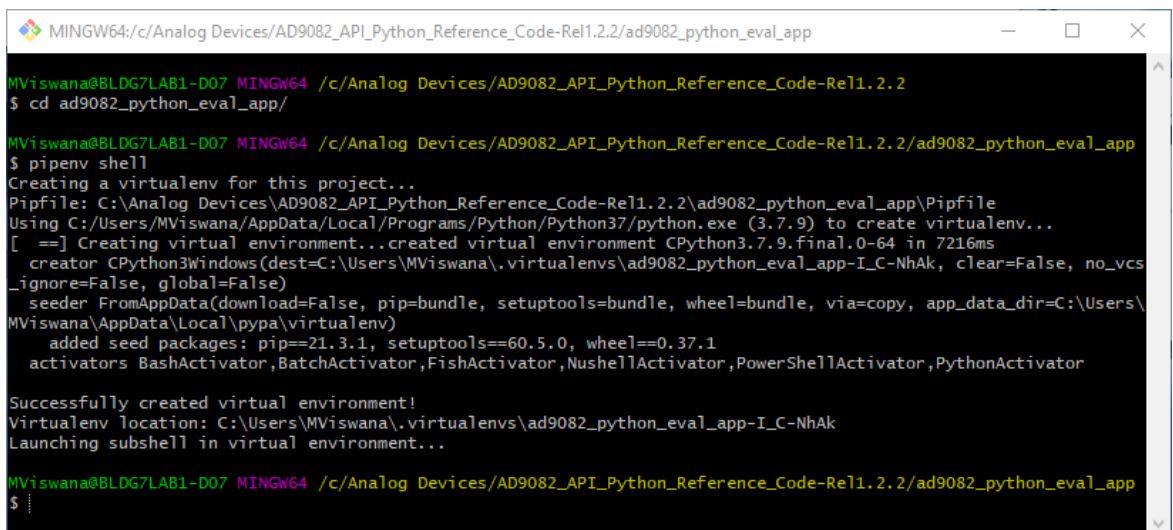
```

MINGW64:/c/Analog Devices/AD9082_API_Python_Reference_Code-Rel1.2.2
MViswana@BLDG7LAB1-D07 MINGW64 /c/Analog Devices/AD9082_API_Python_Reference_Code-Rel1.2.2
$ python --version
Python 3.7.9

MViswana@BLDG7LAB1-D07 MINGW64 /c/Analog Devices/AD9082_API_Python_Reference_Code-Rel1.2.2
$

```

- Navigate to the Python application (adxxxx_python_eval_app) and create a Python Virtual Environment for Python Example App source code package as specified by the "Pipfile".



```

MINGW64:/c/Analog Devices/AD9082_API_Python_Reference_Code-Rel1.2.2/ad9082_python_eval_app
MViswana@BLDG7LAB1-D07 MINGW64 /c/Analog Devices/AD9082_API_Python_Reference_Code-Rel1.2.2/ad9082_python_eval_app
$ cd ad9082_python_eval_app/

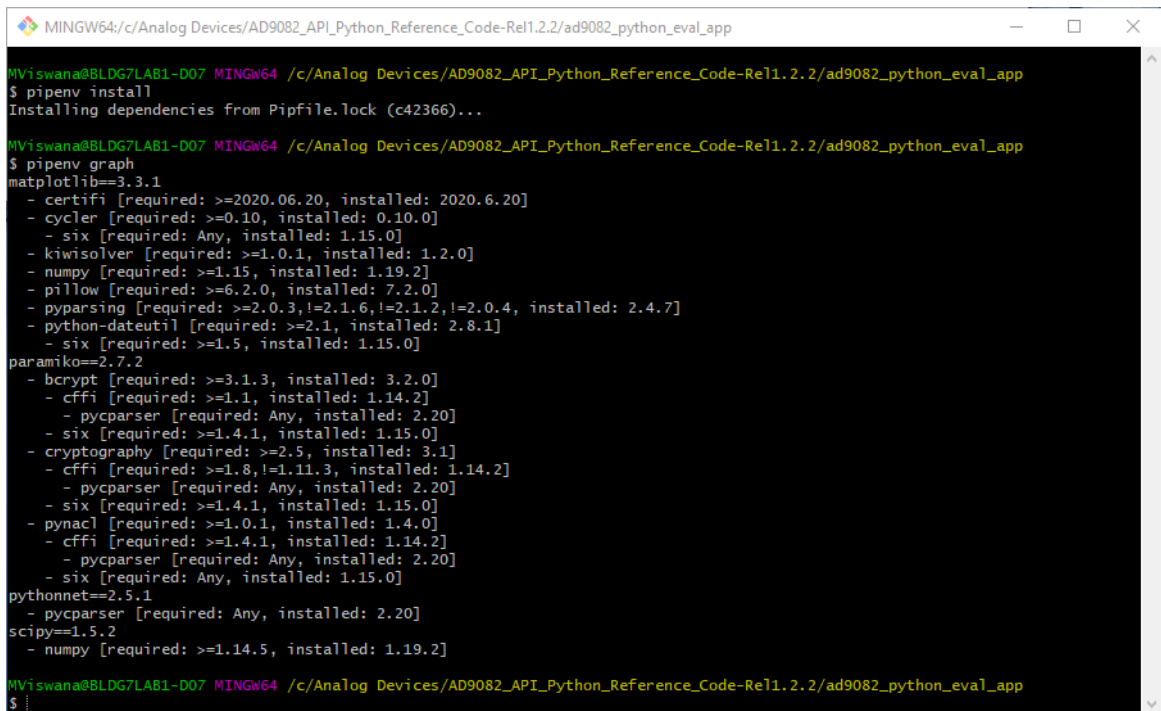
MViswana@BLDG7LAB1-D07 MINGW64 /c/Analog Devices/AD9082_API_Python_Reference_Code-Rel1.2.2/ad9082_python_eval_app
$ pipenv shell
Creating a virtualenv for this project...
Pipfile: C:\Analog Devices\AD9082_API_Python_Reference_Code-Rel1.2.2\ad9082_python_eval_app\Pipfile
Using C:\Users\MViswana\AppData\Local\Programs\Python\Python37\python.exe (3.7.9) to create virtualenv...
[ ==] Creating virtual environment...created virtual environment CPython3.7.9.final.0-64 in 7216ms
creator CPython3Windows(dest=C:\Users\MViswana\.virtualenvs\ad9082_python_eval_app-I_C-NhAk, clear=False, no_vcs
_ignore=False, global=False)
seeder FromAppData(download=False, pip=bundle, setuptools=bundle, wheel=bundle, via=copy, app_data_dir=C:\Users\
MViswana\AppData\Local\pypa\virtualenv)
added seed packages: pip==21.3.1, setuptools==60.5.0, wheel==0.37.1
activators BashActivator,BatchActivator,FishActivator,NushellActivator,PowerShellActivator,PythonActivator

Successfully created virtual environment!
Virtualenv location: C:\Users\MViswana\.virtualenvs\ad9082_python_eval_app-I_C-NhAk
Launching subsHELL in virtual environment...

MViswana@BLDG7LAB1-D07 MINGW64 /c/Analog Devices/AD9082_API_Python_Reference_Code-Rel1.2.2/ad9082_python_eval_app
$

```

- Install the AD9081/2 Python Example App python dependency requires as specified by the "Pipfile". Review Packages installed, Python environment is now ready



```
MINGW64:/c:/Analog Devices/AD9082_API_Python_Reference_Code-Rel1.2.2/ad9082_python_eval_app
MViswana@BLDG7LAB1-D07 MINGW64 /c:/Analog Devices/AD9082_API_Python_Reference_Code-Rel1.2.2/ad9082_python_eval_app
$ pipenv install
Installing dependencies from Pipfile.lock (c42366)...

MViswana@BLDG7LAB1-D07 MINGW64 /c:/Analog Devices/AD9082_API_Python_Reference_Code-Rel1.2.2/ad9082_python_eval_app
$ pipenv graph
matplotlib==3.3.1
- certifi [required: >=2020.06.20, installed: 2020.6.20]
- cyclr [required: >=0.10, installed: 0.10.0]
- six [required: Any, installed: 1.15.0]
- kiwisolver [required: >=1.0.1, installed: 1.2.0]
- numpy [required: >=1.15, installed: 1.19.2]
- pillow [required: >=6.2.0, installed: 7.2.0]
- pyparsing [required: >=2.0.3,!=2.1.6,!=2.1.2,!=2.0.4, installed: 2.4.7]
- python-dateutil [required: >=2.1, installed: 2.8.1]
- six [required: >=1.5, installed: 1.15.0]
paramiko==2.7.2
- bcrypt [required: >=3.1.3, installed: 3.2.0]
- cffi [required: >=1.1, installed: 1.14.2]
- pycparser [required: Any, installed: 2.20]
- six [required: >=1.4.1, installed: 1.15.0]
- cryptography [required: >=2.5, installed: 3.1]
- cffi [required: >=1.8,!=1.11.3, installed: 1.14.2]
- pycparser [required: Any, installed: 2.20]
- six [required: >=1.4.1, installed: 1.15.0]
- pynacl [required: >=1.0.1, installed: 1.4.0]
- cffi [required: >=1.4.1, installed: 1.14.2]
- pycparser [required: Any, installed: 2.20]
- six [required: Any, installed: 1.15.0]
pythonnet==2.5.1
- pycparser [required: Any, installed: 2.20]
scipy==1.5.2
- numpy [required: >=1.14.5, installed: 1.19.2]

MViswana@BLDG7LAB1-D07 MINGW64 /c:/Analog Devices/AD9082_API_Python_Reference_Code-Rel1.2.2/ad9082_python_eval_app
$ |
```

5 Configure Example Application For Target Hardware & Support Files

- Configure the Example Application for you desired platform via setting.py

The screenshot shows a Windows Command Prompt window titled "MINGW64; c:/Analog Devices/AD9082_API_Python_Reference_Code-Rel1.2.2/ad9082_python_eval_app". The prompt is "MViswana@BLDG7LAB1-D07 MINGW64 /c/Analog Devices/AD9082_API_Python_Reference_Code-Rel1.2.2/ad9082_python_eval_app". The user has entered the command "ls", and the output lists the following files and directories:

```
Pipfile
Pipfile.lock
ad9082_client.py
analysis.py
capture_deframer.py
clock_config_factory.py
configure_full_chip_mode.py
configure_nco_test_mode.py
dataI.txt
dataQ.txt
device_config_factory.py
dpg.py
eval_board_target.py
nco_mode.py
settings.py
txfe_dpg_wrapper.py
txfe_ext_ads9_clocking.py
txfe_hmc_ads9_clocking.py
txfe_main.py
use_case_0.py
use_case_10.py
use_case_100_full_rate.py
use_case_11.py
use_case_20.py
use_case_3.py
use_case_33.py
use_case_factory.py
vector_gen.py
```

- Open settings.py with an editor of your choice.
 - This file is where you can configure the application
 - Review Customer Configurable Settings section
 - Clock Configuration
 - All other setting should be left as default as per ADI release package for correct operation with ADI AD9081/AD9082 Evaluation Platform

```

#Customer Configurable Settings
device = "ad9082"

#Desired Clocking Scheme:
use_7044 = True           # Use HMC7044 to generate reference clocks
enable_pll = True         # Use AD9081/2 On-chip PLL to generate ADC & DAC Clocks
hmc7044_crystal_freq = 100.00e6 # 122.88MHz for EBZ-W2 eval boards. 100.00MHz for EBZ-A2

#ADI Platform Recommended Settings
#Default Settings for ADI AD9081/2 Evaluation Platform - Do Not adjust for Correct Operation with ADI Platforms
#ADS Microzed IP Address
ip_address = "192.168.0.10"
eval_board_type = "ce" # "pe" or "ce"

# Folder locations
erpc_folder = r'..\erpc_pkgs' # erpc client/server
fpga_folder = r'..\fpga_images' # FPGA images for PE and CE boards
lib_folder = r'..\data_control_libs' # .net dlls (e.g. DPG)

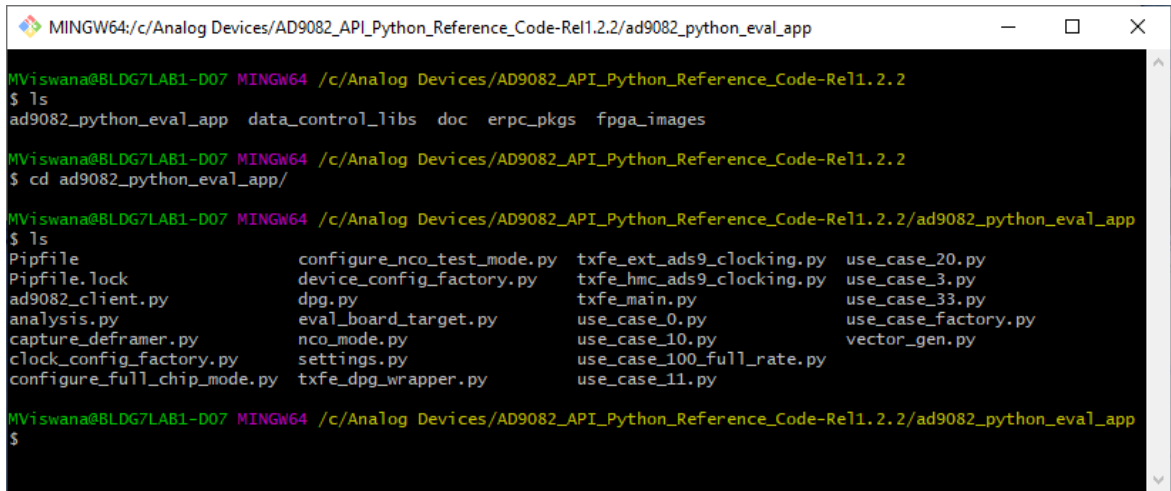
# FPGA Images
fpga_ce = r'ad9986_ads9v2_fmc_19_06_11.bin'
fpga_pe = r'ad9986_ads9v2_evb_kaung.bin'
always_load_fpga_image = False # If True, fpga image is always loaded at startup. If False, will only load image if one not detected.

def info():
    """Prints summary information about the settings"""
    return 'eval_board_type: {}'.format(eval_board_type) + \
        ' use_7044: {}'.format(use_7044) + \
        ' hmc7044_crystal_freq: {} MHz'.format(hmc7044_crystal_freq/1e6) + \
        ' enable_pll: {}'.format(enable_pll)

```

6 Run Example Application

- Ensure you are in Main Source Code Directory.



```

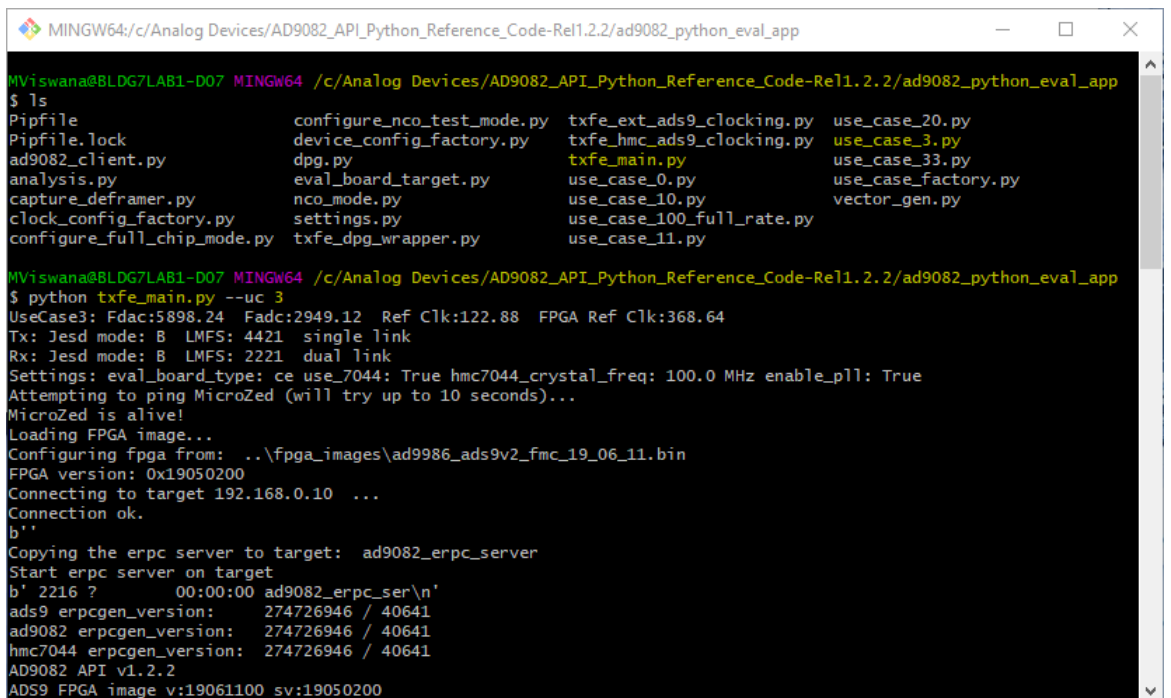
MINGW64:/c:/Analog Devices/AD9082_API_Python_Reference_Code-Rel1.2.2/ad9082_python_eval_app
$ ls
ad9082_python_eval_app  data_control_libs  doc  erpc_pkgs  fpga_images

MINGW64:/c:/Analog Devices/AD9082_API_Python_Reference_Code-Rel1.2.2/ad9082_python_eval_app
$ cd ad9082_python_eval_app/

MINGW64:/c:/Analog Devices/AD9082_API_Python_Reference_Code-Rel1.2.2/ad9082_python_eval_app
$ ls
Pipfile                                configure_nco_test_mode.py  txfe_ext_ads9_clocking.py  use_case_20.py
Pipfile.lock                          device_config_factory.py   txfe_hmc_ads9_clocking.py  use_case_3.py
ad9082_client.py                      dpg.py                     txfe_main.py               use_case_33.py
analysis.py                           eval_board_target.py       use_case_0.py              use_case_factory.py
capture_deframer.py                   nco_mode.py               use_case_10.py             vector_gen.py
clock_config_factory.py               settings.py                use_case_100_full_rate.py
configure_full_chip_mode.py           txfe_dpg_wrapper.py       use_case_11.py

```

- Run the Python example application with an example use-case, this is done using, "txfe_main.py", the top level python application.



```

MINGW64:/c:/Analog Devices/AD9082_API_Python_Reference_Code-Rel1.2.2/ad9082_python_eval_app
$ ls
Pipfile                                configure_nco_test_mode.py  txfe_ext_ads9_clocking.py  use_case_20.py
Pipfile.lock                          device_config_factory.py   txfe_hmc_ads9_clocking.py  use_case_3.py
ad9082_client.py                      dpg.py                     txfe_main.py               use_case_33.py
analysis.py                           eval_board_target.py       use_case_0.py              use_case_factory.py
capture_deframer.py                   nco_mode.py               use_case_10.py             vector_gen.py
clock_config_factory.py               settings.py                use_case_100_full_rate.py
configure_full_chip_mode.py           txfe_dpg_wrapper.py       use_case_11.py

MINGW64:/c:/Analog Devices/AD9082_API_Python_Reference_Code-Rel1.2.2/ad9082_python_eval_app
$ python txfe_main.py --uc 3
UseCase3: Fdac:5898.24  Fadc:2949.12  Ref Clk:122.88  FPGA Ref Clk:368.64
Tx: Jesd mode: 8  LMFS: 4421  single link
Rx: Jesd mode: 8  LMFS: 2221  dual link
Settings: eval_board_type: ce use_7044: True hmc7044_crystal_freq: 100.0 MHz enable_pll: True
Attempting to ping MicroZed (will try up to 10 seconds)...
MicroZed is alive!
Loading FPGA image...
Configuring fpga from: ..\fpga_images\ad9986_ads9v2_fmc_19_06_11.bin
FPGA version: 0x19050200
Connecting to target 192.168.0.10 ...
Connection ok.
b''
Copying the erpc server to target: ad9082_erpc_server
Start erpc server on target
b' 2216 ?      00:00:00 ad9082_erpc_ser\n'
ads9 erpcgen_version: 274726946 / 40641
ad9082 erpcgen_version: 274726946 / 40641
hmc7044 erpcgen_version: 274726946 / 40641
AD9082 API v1.2.2
ADS9 FPGA image v:19061100 sv:19050200

```

- An optional command can be used to run use cases that do not have virtual converter values (jesd M) > 1

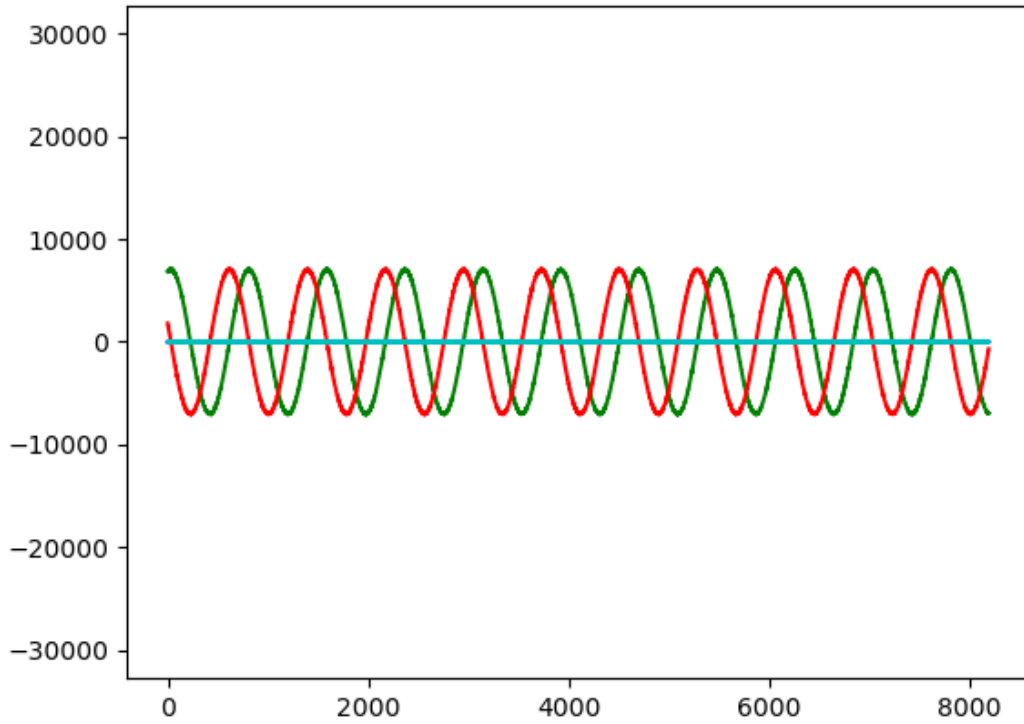
```

$ python txfe_main.py --uc 3 --realonly 1
UseCase3: Fdac:5898.24 Fadc:2949.12 Ref Clk:122.88 FPGA Ref Clk:368.64
Tx: Jesd mode: B LMFS: 4421 single link
Rx: Jesd mode: B LMFS: 2221 dual link
Settings: eval_board_type: ce use_7044: True hmc7044_crystal_freq: 100.0 MHz enable_pll: True
Attempting to ping MicroZed (will try up to 10 seconds)...
MicroZed is alive!
FPGA version: 0x22052600
Connecting to target 192.168.0.10 ...
Connection ok.
b''
Copying the erpc server to target: ad9082_erpc_server
Start erpc server on target
b' 3391 ? 00:00:00 ad9082_erpc_ser\n'
ads9 erpcgen_version: 274726946 / 38869
ad9082 erpcgen_version: 274726946 / 38869
hmc7044 erpcgen_version: 274726946 / 38869
AD9082 API v1.5.0
ADS9 FPGA image v:22052600 sv:22052600
Initializing clocks.
Configure hmc7044
Serial Lane Rate is 7372.8 Gbps
FPGA ref clock 368.64 MHz
HMC7044: PLL is locked
hmc7044 chip id (type, id, grade, rev): 0x0 0x452 0x0 0x1
Initializing clocks done.
Initializing device.
txfe PLL locked
JESD PLL lock status: Locked
ad9082 chip id (type, id, grade, rev): 0xf 0x9082 0x2 0x3
Initializing device done.
ft= 100.0125
Downloading vector to ads9: Single Link
@download_vectors to addr 0x80000000
@download_vectors done.
JRx (DAC) link0 status 0xf
JTx (ADC) link0 status 0x7d
JTx (ADC) link1 status 0x7d
JRx (DAC) link0 is UP!

```

- On a successful completion of a use case you should see an output on the DAC outputs on the board and plot captured by Python.

Figure 1 (Not Responding)



7 Python Example Application- Query APIs

7.1 Overview of sample Debug Scripts

This section describes how to create python scripts to query the evaluation platform using the ADI APIs.

In the Python Eval App folder there is a folder with some example debug scripts. This can be used as an example to build your own scripts.

Note: Run the txfe_main.py application first as it sets up the board and debug environment.

```
NNicheal@NNICHEAL-T01 MINGW64 ~/Txfe_dev/txfedemoapppy/python_eval_app (develop)
$ ls
ad9082_client.py  clock_config_factory.py  data1.txt  device_config_factory.py  nco_mode.py  reg_dump.csv  txfe_ext_ads9_clocking.py  txfe_reg_scratch.py  use_case_100_full_rate.py  use_case_3.py
analysis.py       configure_full_chip_mode.py  data0.txt  dpg.py  Pipfile  settings.py  txfe_hmc_ads9_clocking.py  use_case_0.py  use_case_11.py  use_case_factory.py
capture_deframer.py  configure_nco_test_mode.py  debug_scripts  eval_board_target.py  Pipfile.lock  txfe_dpg_wrapper.py  txfe_main.py  use_case_10.py  use_case_20.py  vector_gen.py
```

```
NNicheal@NNICHEAL-T01 MINGW64 ~/Txfe_dev/txfedemoapppy/python_eval_app/debug_scripts (develop)
$ ls
txfe_api_query_examples.py  txfe_reg_dump.py  txfe_spi_debug.py
```

Run the scripts from the adxxxx_python_eval_app folder.

```
NNicheal@NNICHEAL-T01 MINGW64 ~/Txfe_dev/txfedemoapppy/python_eval_app (develop)
$ python debug_scripts/txfe_spi_debug.py wr 0xA 0x12
Settings: eval_board_type: ce use_7044: True hmc7044_crystal_freq: 100.0 MHz enable_pll: True
ads9 erpcgen_version: 274726946 / 32632
ad9082 erpcgen_version: 274726946 / 32632
hmc7044 erpcgen_version: 274726946 / 32632
spi_accesswr,0xA
spiwrite success 0xA 0x12
done
```

7.2 Creating your own sample Debug Scripts

In your scripts you must first

- Import the AD9081/AD9082/AD9988/AD9986 Client DLL
- establish a TCP connection to the Server running on the MicroZed. The TCP IP details etc are maintained in settings.py in the Python Example Application.

You may then access the devices that the server supports. In the case of the AD9082/AD9081/AD9988/AD9986 ads9, ad9082, hmc7044

Connecting to Server

```
#
# Copyright (c) 2019 Analog Devices, Inc. All Rights Reserved.
# This software is proprietary to Analog Devices, Inc. and its licensors.
#
```

```

""" Example script to Query the AD9082 via API Calls
"""
#pylint: disable=import-error
from __future__ import print_function
import sys, getopt
import os
import numpy as np
import csv
import inspect

currentdir = os.path.dirname(os.path.abspath(inspect.getfile(inspect.currentframe())))
parentdir = os.path.dirname(currentdir)
sys.path.insert(0, parentdir)

import settings
from ad9082_client import Ad9082Client
from ad9082_erpc import adi_ad9082_dac_channel_select_e, adi_ad9082_jesd_link_select_e, adi_cms_chip_id_t

# Print settings configuration
print("Settings: {}".format(settings.info()))

# Create handles to each device API
ad9082_client = Ad9082Client(settings.ip_address)
ads9, ad9082, hmc7044 = ad9082_client.get_devices()

```

Once the Server connection is connected you can call any API supported by AD9081/AD9082/AD9988/AD9986 Header.

Note you drop the first parameter the device handler as this is handled by the server:

```
int32_t adi_ad9082_device_spi_register_set(adi_ad9082_device_t *device, uint16_t addr, uint8_t data);
```

SPI Reads & Writes

```

#Read/Write to SPI Test Register
#Write to Scratch Register
addr = 0xA
ret_val = ad9082.adi_ad9082_device_spi_register_set(addr, 0xAB)
print("ad9082 register write API", hex(addr), "= 0xAB", 1) )
ret_val, reg_val = ad9082.adi_ad9082_device_spi_register_get(addr, int())
print("ad9082 register read API", hex(addr), "=", hex(reg_val) )

```

Note: As There are multiple blocks ADCs, DACs, DUC, DDCs etc in the AD9081/AD9082 if you are reading or writing to low level registers use the Block Select APIs to enable the blocks before you do low level reads and writes

Selecting DAC Channels

```
#DAC Channel API Select API
```



```
ret_val = ad9082.adi_ad9082_dac_chan_select_set( adi_ad9082_dac_channel_select_e.AD9082_DAC_CH_4 |
adi_ad9082_dac_channel_select_e.AD9082_DAC_CH_5);
```

However, API supported APIs will in general hand the Block Selection etc for you:

Example Gain set API

```
#In General ADI APIs will Handle Block Selection etc for you via the main parameters
#Set Gain Example
#int32_t adi_ad9082_dac_duc_nco_gain_set(adi_ad9082_device_t *device, uint8_t channels, uint16_t gain);
ret_val = ad9082.adi_ad9082_dac_duc_nco_gain_set( adi_ad9082_dac_channel_select_e.AD9082_DAC_CH_1, 300);
```

7.3 Note on Reading Back data from API function in Python Environment via eRPC .NET DLL

In C all ADI APIs return the API Error value, to indicate the successful completion of the API or the error that occurred during execution.

Any data in C is returned by reference via the API function call.

See adi_ad908x.h header for examples such as below

```
adi_hmc7044_device_chip_id_get(inout adi_cms_chip_id_t chip_id) -> int32

int32_t adi_txfe_jesd_rx_config_status_get(adi_txfe_device_t *device, uint8_t *cfg_valid);

int32_t adi_txfe_jesd_rx_sample_prbs_test_result_get(adi_txfe_device_t *device, uint8_t *error_flag,
uint32_t *error_count_i, uint32_t *error_count_q);
```

For each of the APIs listed in the header we generate an IDL file which tells the erpc code Tool the direction of the parameters. This is passed to the erpc generate tool that creates a shim and wrapper code for each function for the client type and language required. which we can then build the client. It also generates the code for serialization for the transport layer.

```
adi_hmc7044_device_api_revision_get(inout uint8 rev_major, inout uint8 rev_minor, inout uint8 rev_rc) ->
int32
adi_ad9988_jesd_rx_config_status_get(inout uint8 cfg_valid) -> int32
adi_ad9988_jesd_rx_phy_prbs_test_result_get(uint8 lane, inout adi_ad9988_prbs_test_t prbs_rx_result) ->
int32
```

So for example in a C# client

```
// Read struct adi_cms_chip_id_t function implementation
```

```

private void read_adi_cms_chip_id_t_struct(AdiBasicCodec codec, ref adi_cms_chip_id_t data)
{
    codec.read(ref data.chip_type);

    codec.read(ref data.prod_id);

    codec.read(ref data.prod_grade);

    codec.read(ref data.dev_revision);
}

// Write struct adi_cms_chip_id_t function implementation
private void write_adi_cms_chip_id_t_struct(AdiBasicCodec codec, adi_cms_chip_id_t data)
{
    codec.write(data.chip_type);

    codec.write(data.prod_id);

    codec.write(data.prod_grade);

    codec.write(data.dev_revision);
}

```

So then in Python

```

return_val, chip_id = txfe.adi_ad9986_device_chip_id_get(adi_cms_chip_id_t())
print("ad9082 chip id (type, id, grade, rev): ", hex(chip_id.chip_type), hex(chip_id.prod_id),
      hex(chip_id.prod_grade), hex(chip_id.dev_revision))

```

8 Customizing or Creating a Use-case

- Each use case is stored in a separate file following the standard "use_case_#.py". To customize these use cases, the selected one can be opened in the editor of your choosing. The file contains a use case object with all the parameters for customization listed within.

Use case Template Example

```
class UseCase(object):
    """ Class for configuring Tx and Rx for use case #3 (JESD204 7.37Gbps)

    """
    def __init__(self, description_formatter = None):
        self.description_formatter
        self.type
        self.name

        ##### Clocks #####

        ##### DAC #####

        ##### ADC #####

        # setup ad9082 jtx converter mapping

    def _jrx_param(self):
        """Setup jrx jesd params (DAC)
        """

        return jrx_param

    def _jtx_param(self):
        """Setup jtx jesd params (ADC)
        """

        return [jtx_param, jtx_param] # Link0, Link1

    def description(self):
        """ Brief description of use case
        """
```

- If creating your own use case, use the provided use cases as a template to define your class and parameters.
- Once your use case is created, add the created use case to the use_case_factory.c file. Here, you will need to add an if statement pointing to your use case as well as add it as an import at the top of the file. If customizing a use case, you can ignore this step.

```

from __future__ import print_function
from use_case_0 import UseCase0 ←
from use_case_3 import UseCase3
from use_case_10 import UseCase10
from use_case_11 import UseCase11
from use_case_20 import UseCase20
from use_case_33 import UseCase33
from use_case_100_full_rate import UseCase100FullRate

```

```

class UseCaseFactory(object):
    """Factory class for creating use case objects
    """

    def __init__(self):
        """Constructor
        """

    def create(self, uc, description_formatter):
        """ Create a use case object from a use case number
        """
        if (uc == 0):
            uc = UseCase0()                    # NCO test mode
        elif (uc == 3):
            uc = UseCase3(description_formatter) # JESD204B at 7.33Gbps lane rate.
        elif uc == 10:
            uc = UseCase10(description_formatter) # JESD204C at 8Gbps lane rate.
        elif uc == 11:
            uc = UseCase11(description_formatter) # JESD204C at 16Gbps lane rate.
        elif uc == 20:
            uc = UseCase20(description_formatter) # JESD204C at 24Gbps lane rate.
        elif uc == 33:
            uc = UseCase33(description_formatter) # JESD204C at 23.76Gbps lane rate.
        else:
            raise Exception("Selected use case #{0} not found".format(uc))

        return uc

```

9 AD9081 eRPC Python Client - Server Demo

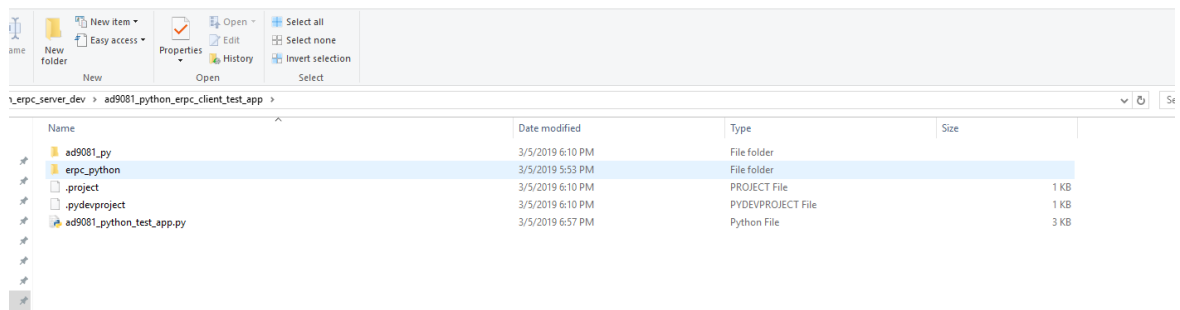
Simple Python Script to demo accessing API from Python via the TxFE eRPC Client Server

9.1 Python with eRPC Client - Server Demo

1. Download example code : [ad9081_python_erpc_client_test_app.zip](https://confluence.analog.com/download/attachments/275123040/ad9081_python_erpc_client_test_app.zip?api=v2&modificationDate=1620228976919&version=1)² and extract to your desire python workspace

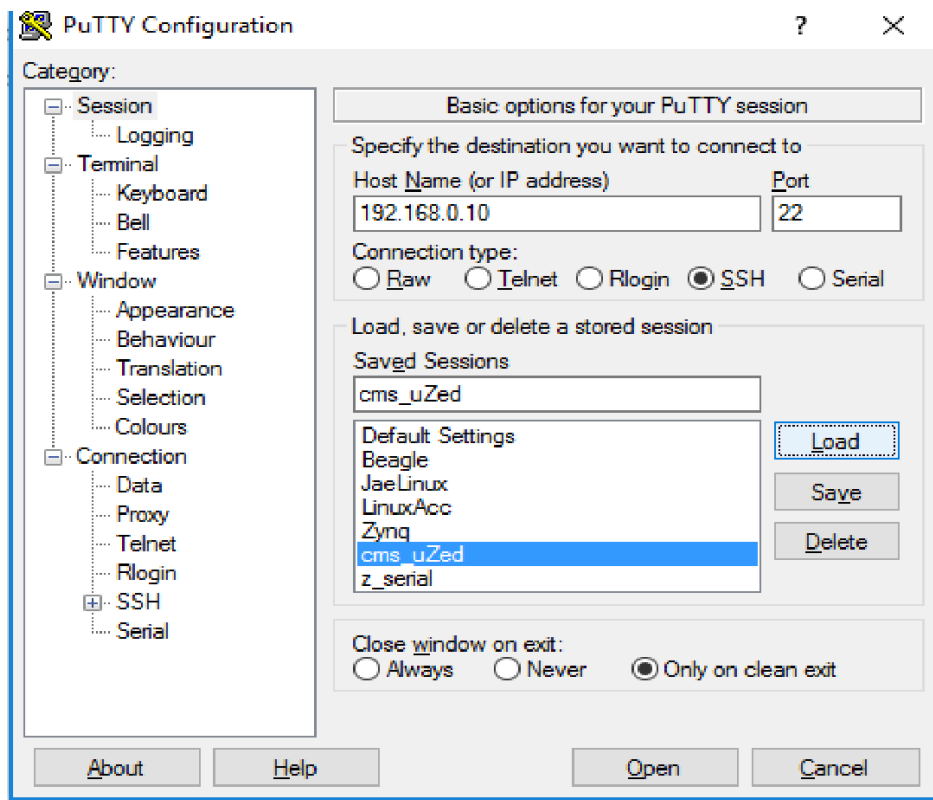
You should see the following Files

- **ad9081_py folder:** The python client class describing the APIs exposed via the TxFE eRPC Client -Server.
- **erpc_python folder:** A erpc protocol files for python client server operation
- **ad9081_python_test_app:** Simple python script using AD9081 eRPC Client Server



2. Power up TxFE Evaluation platform Board. Ensure you have power to your daughter card.
 - You can achieve this via ACE or DPG downloader tool
3. Use Putty to SSH to MZ Board:

² https://confluence.analog.com/download/attachments/275123040/ad9081_python_erpc_client_test_app.zip?api=v2&modificationDate=1620228976919&version=1



4. Run the AD9081 eRPC Server

- run `./ad9081_erpc_server`

```

root@analog: ~
root@analog:~# ./ad9081_erpc_server
Board ID: 0, Build Date: 8b0e1
chip-to-chip bridge: Ready detected.....
Found UIO SPI devices: uio_spi_controller, uio_spi_gpio, uio_axi_fpga.
The UIO SPI mapped address is 0xb6fd4000, 0xb6fd3000, 0xb6fa6000.
in serverThreadStub (arg=0x462c0)
in server thread
Listening for connections

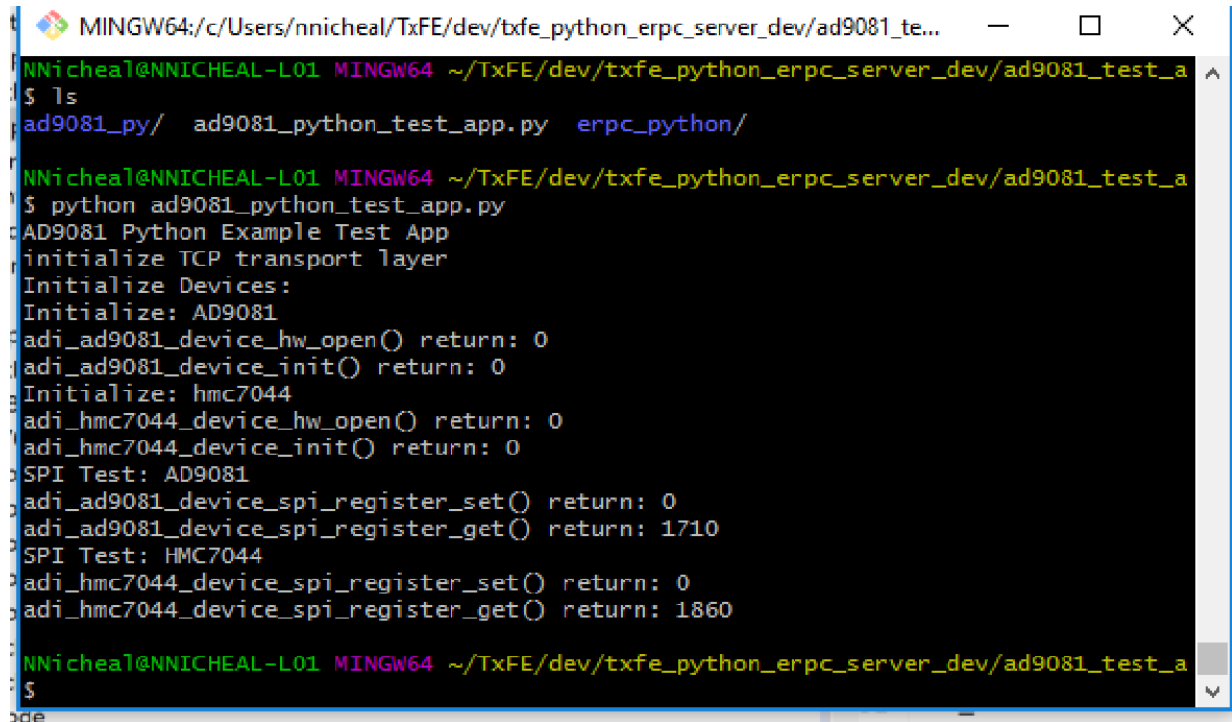
```

- Note you may get Chip To Chip Bridge Error first, (This is known issue), But on second

- You should see that the server is now listening for API Call From PC Client.

5. Using you chosen Python tool, run the example Python Script

- You should see API Call over the client server to the TxFE Device and the Clock Chip Device and do spi read and writes
- Here is an example calling python from command line.



```

MINGW64:/c/Users/nnicheal/TxFE/dev/txfe_python_erpc_server_dev/ad9081_test_a...
NNicheal@NNICHEAL-L01 MINGW64 ~/TxFE/dev/txfe_python_erpc_server_dev/ad9081_test_a
$ ls
ad9081_py/  ad9081_python_test_app.py  erpc_python/

NNicheal@NNICHEAL-L01 MINGW64 ~/TxFE/dev/txfe_python_erpc_server_dev/ad9081_test_a
$ python ad9081_python_test_app.py
AD9081 Python Example Test App
initialize TCP transport layer
Initialize Devices:
Initialize: AD9081
adi_ad9081_device_hw_open() return: 0
adi_ad9081_device_init() return: 0
Initialize: hmc7044
adi_hmc7044_device_hw_open() return: 0
adi_hmc7044_device_init() return: 0
SPI Test: AD9081
adi_ad9081_device_spi_register_set() return: 0
adi_ad9081_device_spi_register_get() return: 1710
SPI Test: HMC7044
adi_hmc7044_device_spi_register_set() return: 0
adi_hmc7044_device_spi_register_get() return: 1860

NNicheal@NNICHEAL-L01 MINGW64 ~/TxFE/dev/txfe_python_erpc_server_dev/ad9081_test_a
$

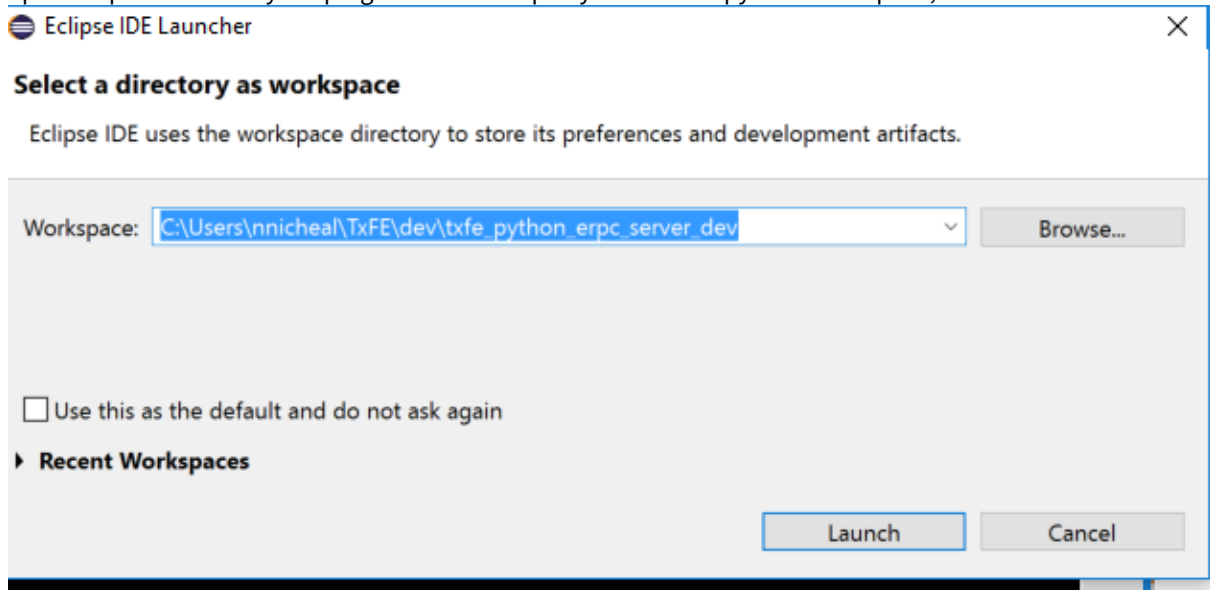
```

9.2 Python with eRPC Client - Server Demo with Eclipse and Pydev

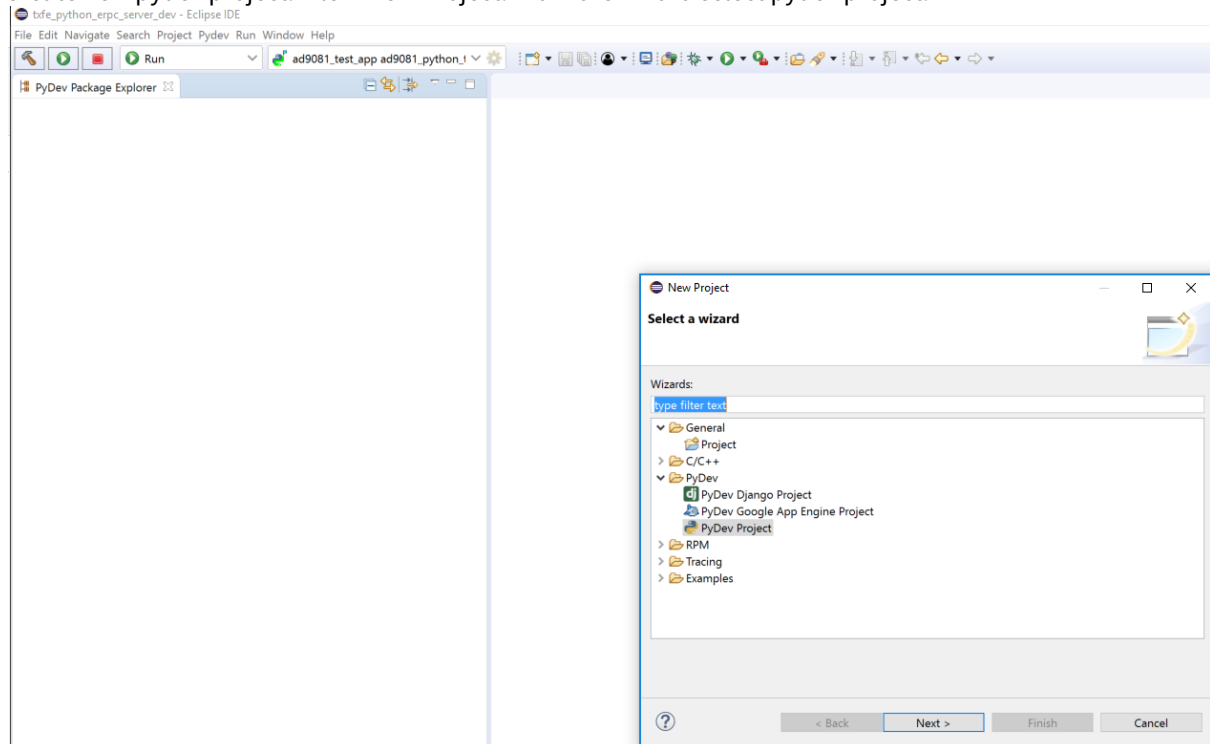
1. Download example code : [ad9081_python_erpc_client_test_app.zip](https://confluence.analog.com/download/attachments/275123040/ad9081_python_erpc_client_test_app.zip)³ and extract to your desire python workspace

³ https://confluence.analog.com/download/attachments/275123040/ad9081_python_erpc_client_test_app.zip?api=v2&modificationDate=1620228976919&version=1

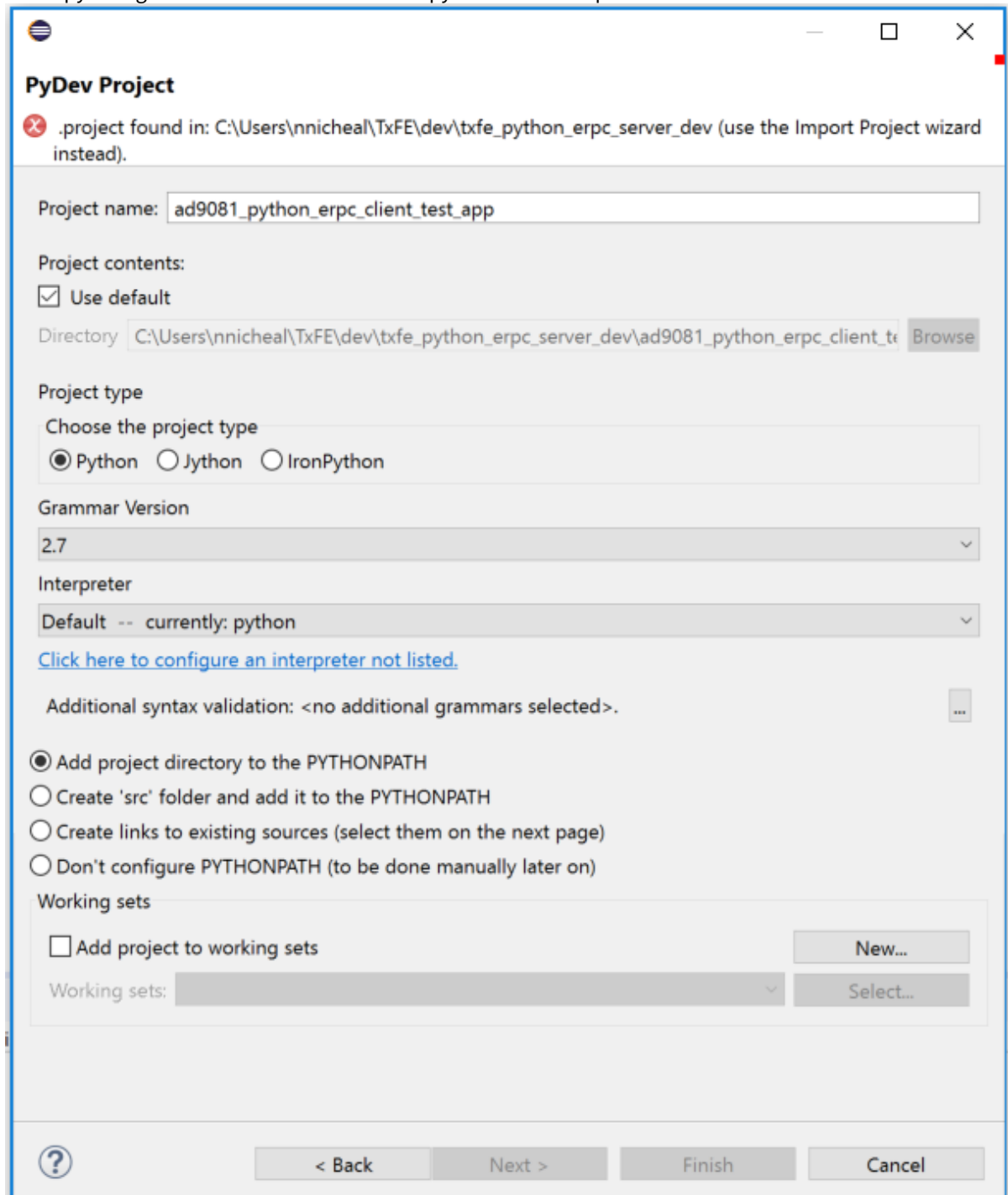
2. Open eclipse IDE with Pydev plugin installed. Open your desired python workspace, ensure



3. Create new pydev project. File → New Project. From the wizard select pydev project.



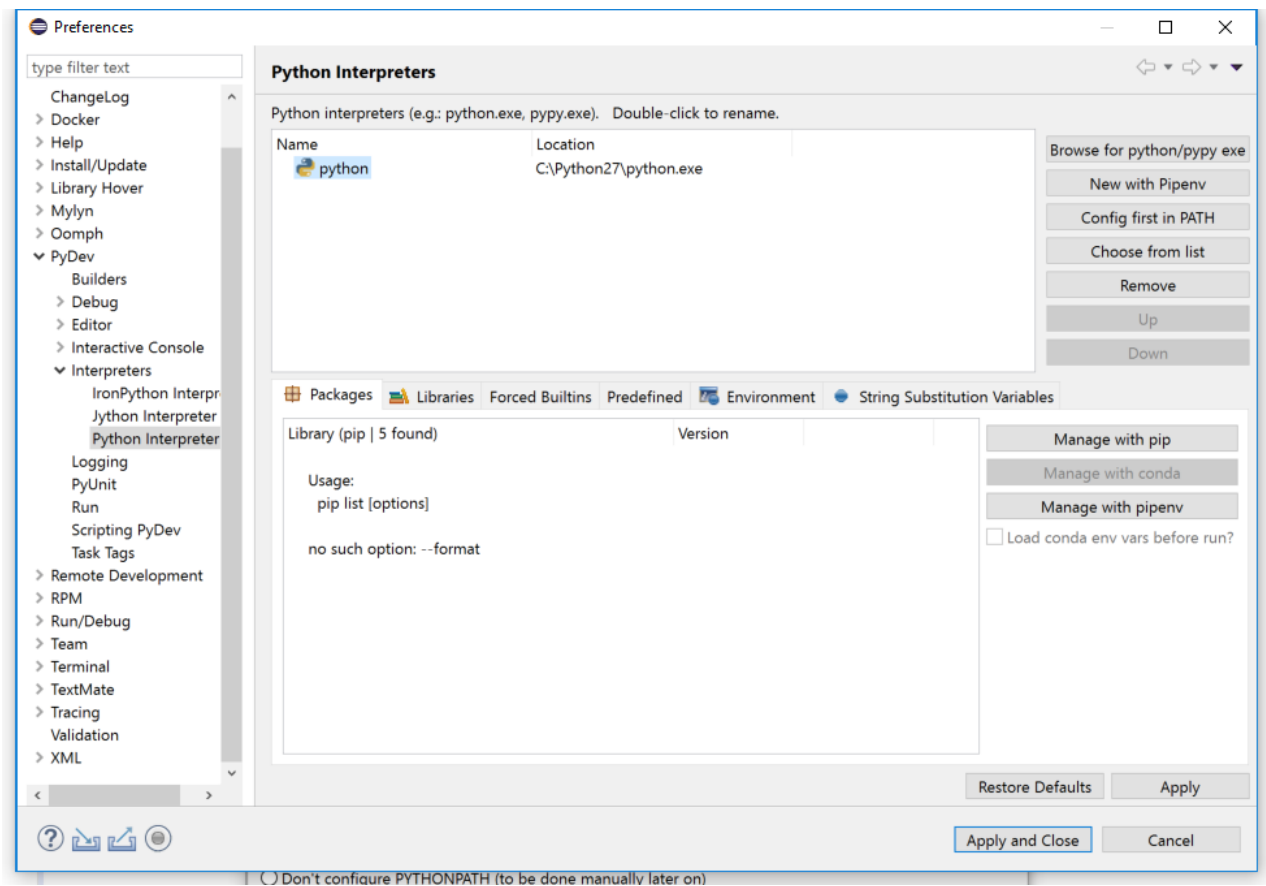
- Set project name as ad9081_python_erpc_client_test_app same as the code provided
Select python grammar version to 2.7 and set pydev as the interpreter



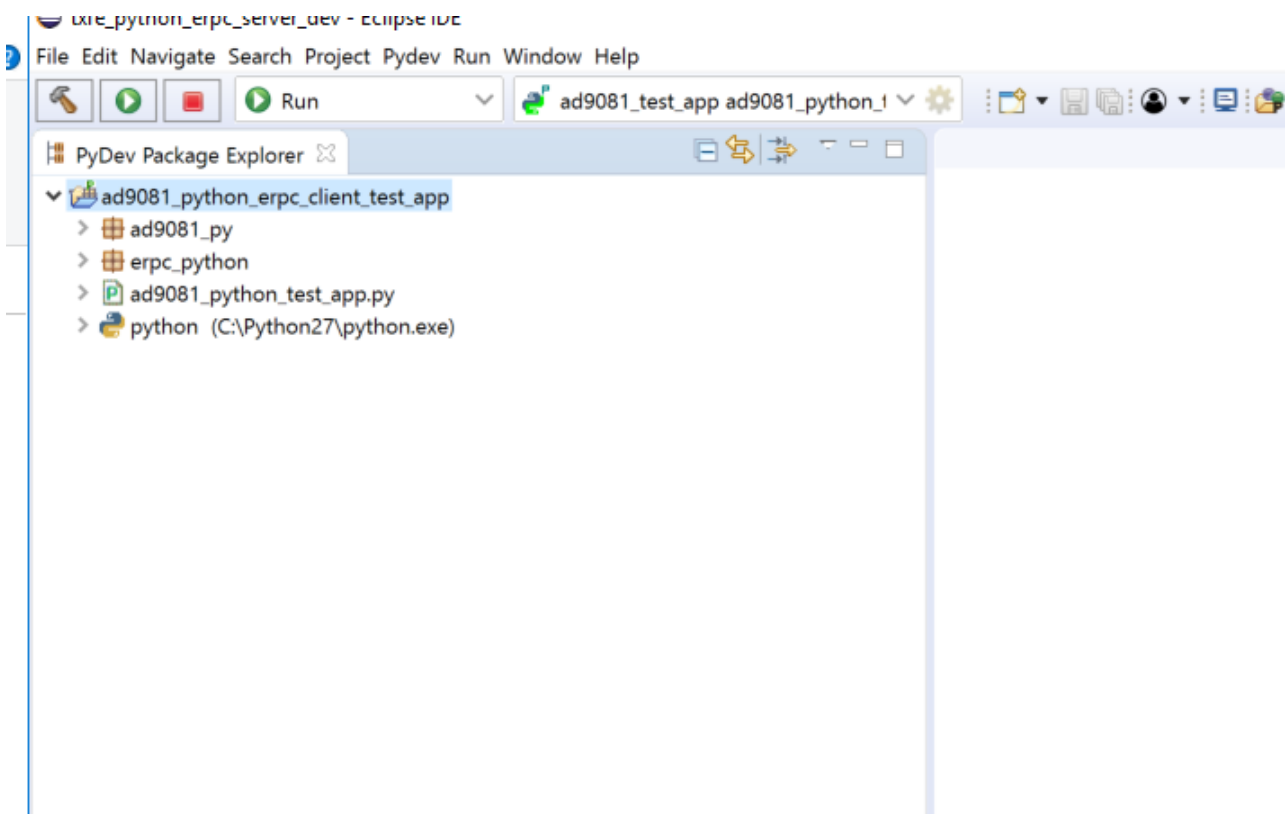
The image shows the 'PyDev Project' dialog box in an IDE. It contains the following fields and options:

- Project name:** A text box containing 'ad9081_python_erpc_client_test_app'.
- Project contents:**
 - ☒ Use default
 - Directory:** A text box showing 'C:\Users\nnicheal\TxFE\dev\txfe_python_erpc_server_dev\ad9081_python_erpc_client_test_app' with a 'Browse' button.
- Project type:**
 - Choose the project type
 - ☒ Python ☐ Jython ☐ IronPython
- Grammar Version:** A dropdown menu showing '2.7'.
- Interpreter:** A dropdown menu showing 'Default -- currently: python'.
- [Click here to configure an interpreter not listed.](#)
- Additional syntax validation:** A text box showing '<no additional grammars selected>'.
- PythonPATH options:**
 - ☒ Add project directory to the PYTHONPATH
 - ☐ Create 'src' folder and add it to the PYTHONPATH
 - ☐ Create links to existing sources (select them on the next page)
 - ☐ Don't configure PYTHONPATH (to be done manually later on)
- Working sets:**
 - ☐ Add project to working sets
 - A 'New...' button.
 - A 'Working sets:' dropdown menu.
 - A 'Select...' button.

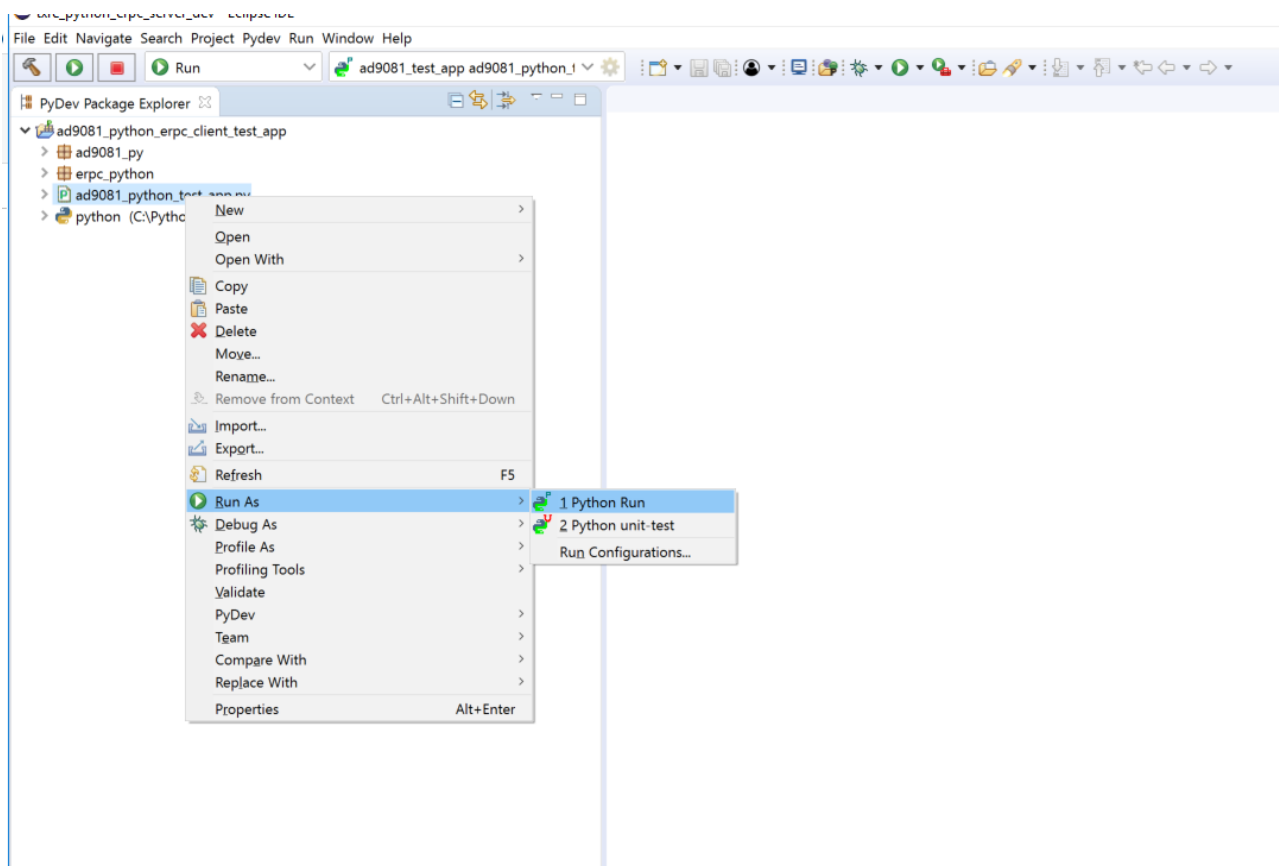
At the bottom, there are navigation buttons: '< Back', 'Next >', 'Finish', and 'Cancel', along with a help icon (?) on the left.



5. Package Explorer Should now display the AD9081 Python Code



6. Right Click on the test app python script and select run as → python



7. You will see the output in the console

